

informações

limitam concorrência, mas todas as t. executam

controle de concorrência

Protocolo de Bloqueio



Tabela de bloqueio registra emissão de solicitação de bloqueios para itens

- REGRAS → impostas pelo gerenciador de bloqueio
 - Transação precisa emitir bloqueio antes de operar sobre item
 - Transação precisa emitir desbloqueio após completar operações sobre o item
 - Transação não emite bloqueio se já tiver bloqueio sobre item
 - PROMOÇÃO (upgrade) → bloqueio $rl(x)$ convertido para $wl(x)$
 - DESCENSO (downgrade) → bloqueio $wl(x)$ convertido para $rl(x)$
 - Transação não emite desbloqueio se não tiver bloqueio sobre item
 - Solicitação de bloqueio não atendida entra em fila de espera por desbloqueio

Aplicação de regras de bloqueio e desbloqueio não garante serialização

Regras adicionais para bloqueios e desbloqueios em fases são necessárias

o que ele gera ou é serial ou é serializável. ou seja, garante isolamento.

Nesse caso, os read locks podem acontecer ao mesmo tempo, mas se alguém está lendo vc não pode escrever

Isso gera maior concorrência

Com um operador binário, temos bloqueio e desbloqueio. Com o ternário, temos read lock, write lock e unlock

Só aplicar essas regras não garante serialização. Por isso, temos o protocolo de bloqueio.

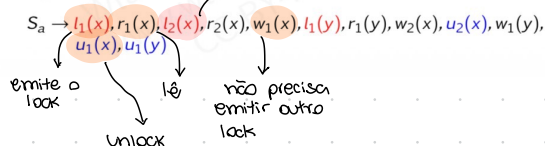
Funciona como um semáforo, bloqueio de passar quando está vermelho

Uma vez que vc bloqueia, só pode desbloquear no final, antes de confirmar.

Expansão → lock, lock, lock...

Retração → unlock, unlock...

Aplicação do protocolo 2PL



$Fb = l_1(Nb), l_1(y)$
(fila de bloqueio)

$Fe = l_2(Nb)$
(fila de espera)

$S_d = r_1(Nb), w_1(Nb), r_1(y), w_1(y), r_2(Nb), w_2(Nb)$

apenas acontece quando é liberado pela t.1

deadlock

Situação de impasse em que uma transação está esperando o desbloqueio de outra, que está esperando o desbloqueio da primeira.

→ SGBD resolve!

Existem 4 formas:

- Alterar o apl. estrito para o conservador ao invés de emitir a medida que precise, emite tudo antes. *limita a conc. mas não tem deadlock*
- Instituir a figura de um assassino usar a d. de nasc. (timestamp) de uma transação para definir. A mais nova será abortada e executada no próx. escalonamento. *quem nasce 1º é quem começa a executar 1º*

Obs: SGBD não usa uma única técnica, usa várias.

O problema do deadlock é um assassino perseguidor, que sempre mata o mesmo cara e ele nunca executa → starvation. A solução é a manutenção do timestamp

03 → Grafo de espera

- DETECÇÃO → transações em impasse, mais nova aborta e é reescalada
- Transação mais nova, provavelmente com menos operações executadas
- GRAFO DE ESPERA → nós são transações, arestas direcionadas são bloqueios em espera, ciclo indica impasse



$S_a \rightarrow r_1(x), r_2(x), w_1(x), r_1(y), w_2(x), w_1(y)$

Ciclo entre T_1 e T_2 , T_2 aborta e é reescalada por ser a mais nova

Se der ciclo, alguém tem que morrer. Normalmente, é a mais nova

04 → PRESUNÇÃO → transação demorada aborta e é reescalada

- TEMPO LIMITE (timeout) → registro de tempo
- Presume-se impasse na transação se seu tempo de execução exceder timeout
- Simples, evita sobrecarga